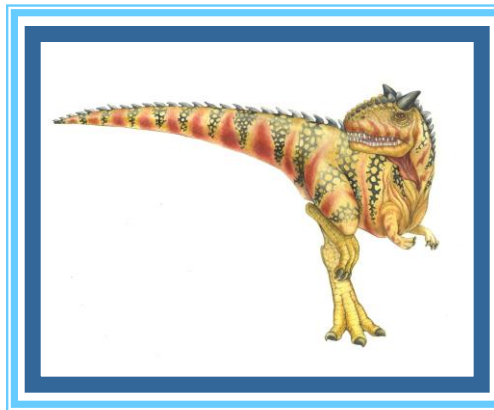


Capítulo 4: Tópicos





Capítulo 4: Tópicos

- ❑ Visão geral
- ❑ Programação Multicore
- ❑ Modelos multithreading
- ❑ Bibliotecas de Tópicos
- ❑ Encadeamento implícito
- ❑ Problemas de encadeamento
- ❑ Exemplos de sistemas operacionais





Objetivos

- ❑ Para introduzir a noção de thread — uma unidade fundamental de utilização da CPU que forma a base de sistemas de computadores multithread
- ❑ Para discutir as APIs para as bibliotecas de threads Pthreads, Windows e Java
- ❑ Para explorar diversas estratégias que fornecem encadeamento implícito
- ❑ Para examinar questões relacionadas à programação multithread
- ❑ Para cobrir o suporte do sistema operacional para threads no Windows e Linux





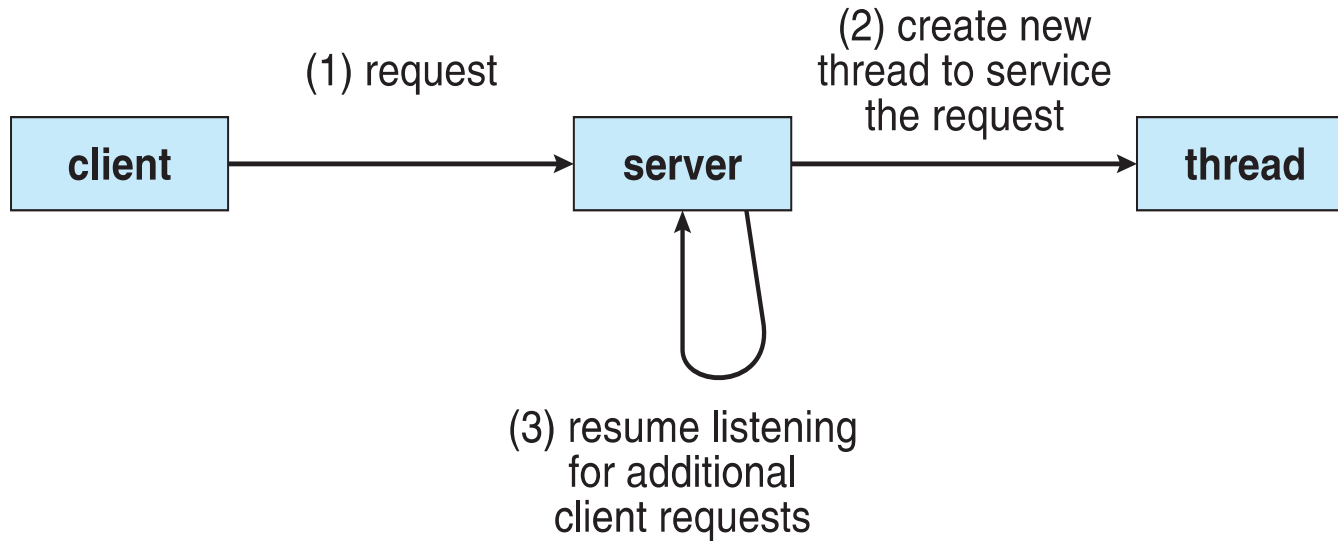
Motivação

- ❑ A maioria dos aplicativos modernos são multithread
- ❑ Os threads são executados dentro do aplicativo
- ❑ Várias tarefas com o aplicativo podem ser implementadas por threads separados
 - ❑ Atualizar exibição
 - ❑ Buscar dados
 - ❑ Verificação ortográfica
 - ❑ Responder a uma solicitação de rede
- ❑ A criação de processos é pesada, enquanto a criação de threads é leve
- ❑ Pode simplificar o código e aumentar a eficiência
- ❑ Os kernels são geralmente multithread





Arquitetura de servidor multithread





Benefícios

- ❑ **Capacidade de resposta** – pode permitir a execução contínua se parte do processo estiver bloqueada, especialmente importante para interfaces de usuário
- ❑ **Compartilhamento de recursos** – threads compartilham recursos de processo, mais fácil do que memória compartilhada ou passagem de mensagens
- ❑ **Economia** – mais barato que a criação de processos, troca de threads com menor sobrecarga que a troca de contexto
- ❑ **Escalabilidade** – o processo pode tirar proveito das arquiteturas multiprocessadoras





Programação Multicore

- **multicore** ou **multiprocessadores** colocam pressão sobre os programadores, os desafios incluem:
 - **Dividindo atividades**
 - **Equilíbrio**
 - **Divisão de dados**
 - **Dependência de dados**
 - **Teste e depuração**
- **O paralelismo** implica que um sistema pode executar mais de uma tarefa simultaneamente
- **A simultaneidade** oferece suporte a mais de uma tarefa para fazer progresso
 - Processador único/núcleo, planejador fornecendo simultaneidade





Programação Multicore (Cont.)

- Tipos de paralelismo
 - **Paralelismo de dados** – distribui subconjuntos dos mesmos dados em vários núcleos, mesma operação em cada
 - **Paralelismo de tarefas** – distribuição de threads entre núcleos, cada thread executando uma operação única
- À medida que o número de threads cresce, também aumenta o suporte arquitetônico para threading
 - As CPUs têm núcleos e também ***threads de hardware***
 - Considere o Oracle SPARC T4 com 8 núcleos e 8 threads de hardware por núcleo



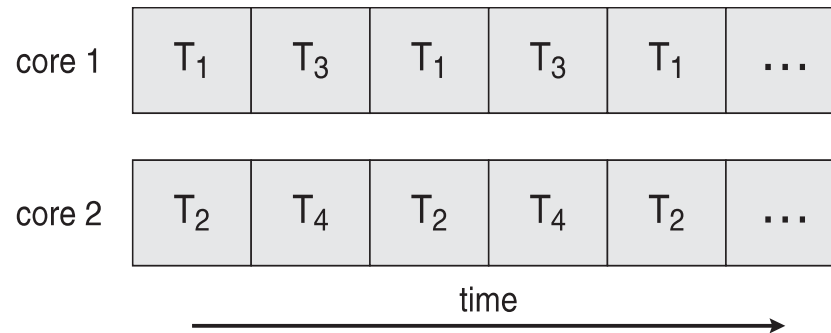


Concorrência vs. Paralelismo

❑ Execução simultânea em sistema single-core:

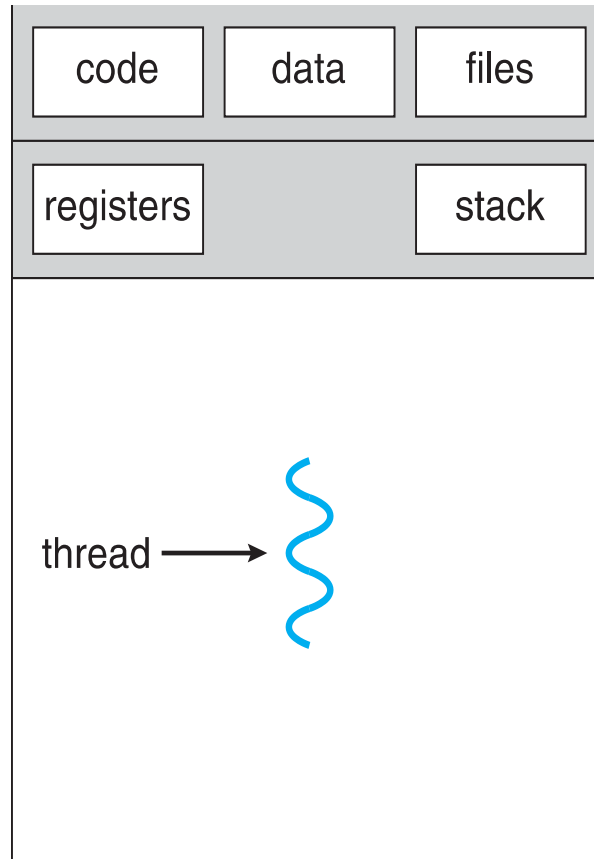


❑ Paralelismo em um sistema multi-core:

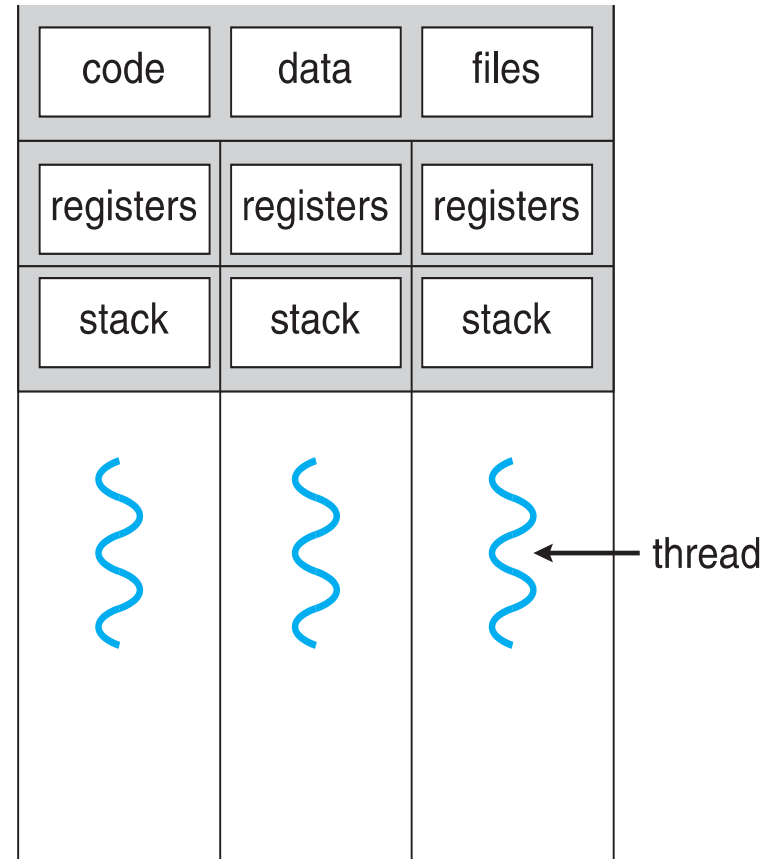




Processos Single e Multithreaded



single-threaded process



multithreaded process





Lei de Amdahl

- ❑ Identifica ganhos de desempenho ao adicionar núcleos adicionais a um aplicativo que possui componentes seriais e paralelos
- ❑ S é a porção serial
- ❑ N núcleos de processamento

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- ❑ Ou seja, se a aplicação for 75% paralela / 25% serial, mover de 1 para 2 núcleos resulta em aceleração de 1,6 vezes
- ❑ À medida que N se aproxima do infinito, a aceleração se aproxima de $1 / S$

A parte serial de um aplicativo tem efeito desproporcional no desempenho obtido pela adição de núcleos adicionais

- ❑ Mas a lei leva em consideração os sistemas multicore contemporâneos?





Threads do usuário e threads do kernel

- **Threads do usuário** - gerenciamento feito pela biblioteca de threads em nível de usuário
- Três bibliotecas de threads principais:
 - **Tópicos**
 - Tópicos do Windows
 - Tópicos Java
- **Threads do kernel** - Suportado pelo Kernel
- Exemplos – praticamente todos os sistemas operacionais de uso geral, incluindo:
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X





Modelos multithreading

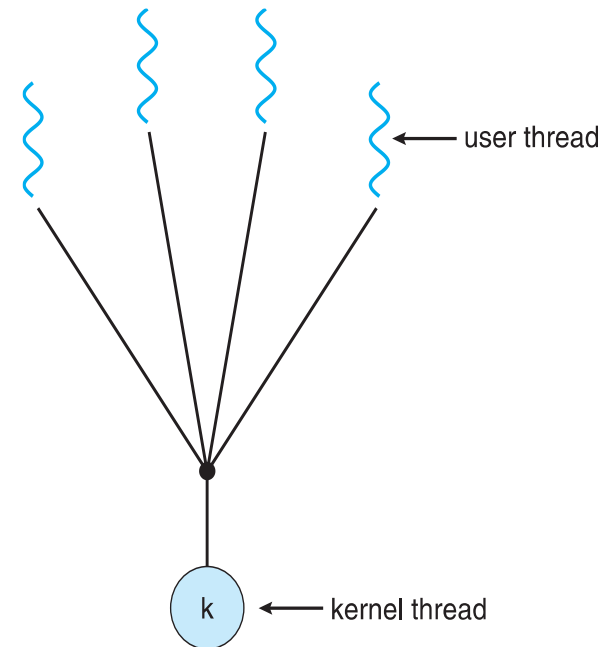
- Muitos para um
- Um para um
- Muitos para muitos





Muitos para um

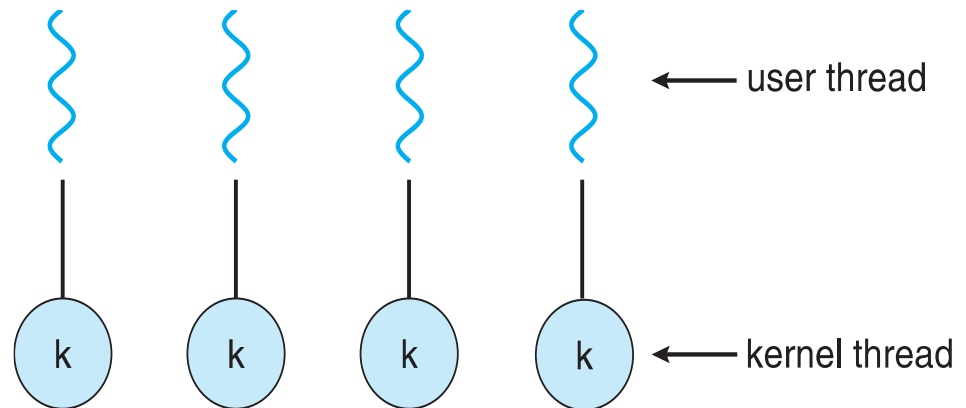
- ❑ Muitos threads de nível de usuário mapeados para um único thread de kernel
- ❑ Um bloqueio de thread faz com que todos bloqueiem
- ❑ Vários threads não podem ser executados em paralelo em um sistema multicore porque apenas um pode estar no kernel por vez
- ❑ Poucos sistemas atualmente usam esse modelo
- ❑ Exemplos:
 - ❑ **Fios Verdes Solaris**
 - ❑ **Tópicos Portáteis GNU**





Um para um

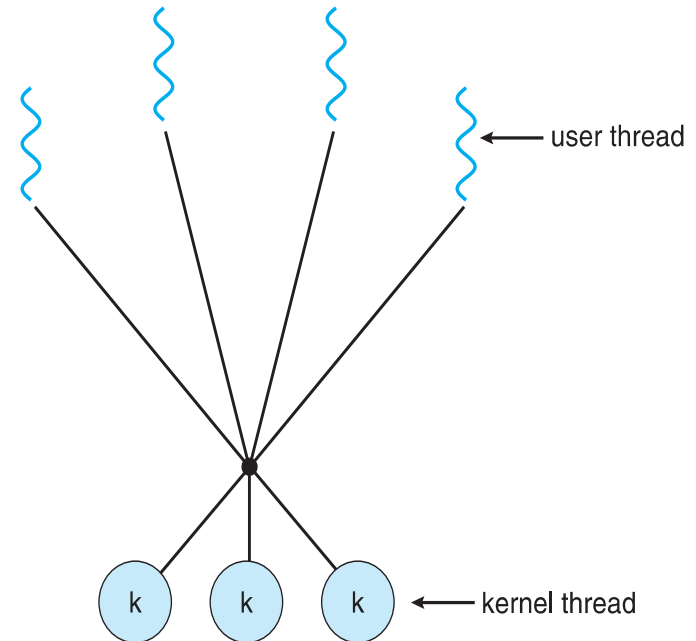
- ❑ Cada thread de nível de usuário é mapeado para thread do kernel
- ❑ A criação de um thread de nível de usuário cria um thread de kernel
- ❑ Mais simultaneidade do que muitos para um
- ❑ Número de threads por processo às vezes restrito devido à sobrecarga
- ❑ Exemplos
 - ❑ Windows
 - ❑ Linux
 - ❑ Solaris 9 e posterior





Modelo muitos para muitos

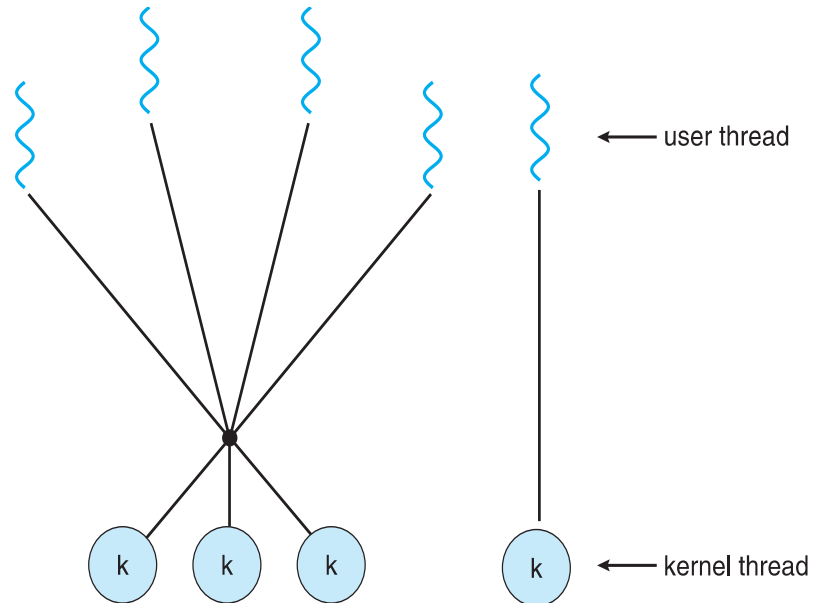
- ❑ Permite que muitos threads de nível de usuário sejam mapeados para muitos threads de kernel
- ❑ Permite que o sistema operacional crie um número suficiente de threads de kernel
- ❑ Solaris antes da versão 9
- ❑ Windows com o pacote *ThreadFiber*





Modelo de dois níveis

- Semelhante ao M:M, exceto que permite que um thread do usuário seja **vinculado** ao thread do kernel
- Exemplos
 - IRIX
 - HP-UX
 - Tru64 UNIX
 - Solaris 8 e anteriores





Bibliotecas de Tópicos

- **Biblioteca de tópicos** fornece ao programador uma API para criar e gerenciar threads
- Duas formas principais de implementação
 - Biblioteca inteiramente no espaço do usuário
 - Biblioteca de nível de kernel suportada pelo sistema operacional





Tópicos P

- ❑ Pode ser fornecido em nível de usuário ou em nível de kernel
- ❑ Uma API padrão POSIX (IEEE 1003.1c) para criação e sincronização de threads
- ❑ **Especificação** , não **implementação**
- ❑ A API especifica o comportamento da biblioteca de threads, a implementação depende do desenvolvimento da biblioteca
- ❑ Comum em sistemas operacionais UNIX (Solaris, Linux, Mac OS X)





Exemplo de Pthreads

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```





Exemplo de Pthreads (continuação)

```
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid,&attr,runner,argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

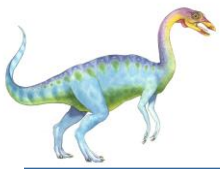
    printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```





Código Pthreads para juntar 10 threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```





Programa C multithread do Windows

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* the thread runs in this separate function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 0; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    if (argc != 2) {
        fprintf(stderr, "An integer parameter is required\n");
        return -1;
    }
    Param = atoi(argv[1]);
    if (Param < 0) {
        fprintf(stderr, "An integer >= 0 is required\n");
        return -1;
    }
}
```





Programa C multithread do Windows (continuação)

```
/* create the thread */
ThreadHandle = CreateThread(
    NULL, /* default security attributes */
    0, /* default stack size */
    Summation, /* thread function */
    &Param, /* parameter to thread function */
    0, /* default creation flags */
    &ThreadId); /* returns the thread identifier */

if (ThreadHandle != NULL) {
    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}
}
```





Tópicos Java

- ❑ Os threads Java são gerenciados pela JVM
- ❑ Normalmente implementado usando o modelo de threads fornecido pelo sistema operacional subjacente
- ❑ Os threads Java podem ser criados por:

```
public interface Runnable
{
    public abstract void run();
}
```

- ❑ Estendendo a classe Thread
- ❑ Implementando a interface Runnable





Programa Java Multithread

```
class Sum
{
    private int sum;

    public int getSum() {
        return sum;
    }

    public void setSum(int sum) {
        this.sum = sum;
    }
}

class Summation implements Runnable
{
    private int upper;
    private Sum sumValue;

    public Summation(int upper, Sum sumValue) {
        this.upper = upper;
        this.sumValue = sumValue;
    }

    public void run() {
        int sum = 0;
        for (int i = 0; i <= upper; i++)
            sum += i;
        sumValue.setSum(sum);
    }
}
```





Programa Java Multithread (Cont.)

```
public class Driver
{
    public static void main(String[] args) {
        if (args.length > 0) {
            if (Integer.parseInt(args[0]) < 0)
                System.err.println(args[0] + " must be >= 0.");
            else {
                Sum sumObject = new Sum();
                int upper = Integer.parseInt(args[0]);
                Thread thrd = new Thread(new Summation(upper, sumObject));
                thrd.start();
                try {
                    thrd.join();
                    System.out.println
                        ("The sum of "+upper+" is "+sumObject.getSum());
                } catch (InterruptedException ie) { }
            }
        }
        else
            System.err.println("Usage: Summation <integer value>");
    }
}
```





Encadeamento implícito

- Crescendo em popularidade à medida que o número de threads aumenta, a correção do programa é mais difícil com threads explícitos
- Criação e gerenciamento de threads feitos por compiladores e bibliotecas de tempo de execução em vez de programadores
- Três métodos explorados
 - Pools de Tópicos
 - OpenMP
 - Despacho Grand Central
- Outros métodos incluem Microsoft Threading Building Blocks (TBB), pacote `java.util.concurrent`





Pools de Tópicos

- ❑ Crie uma série de threads em um pool onde eles aguardam trabalho
- ❑ Vantagens:
 - ❑ Geralmente é um pouco mais rápido atender a uma solicitação com um thread existente do que criar um novo thread
 - ❑ Permite que o número de threads no(s) aplicativo(s) seja vinculado ao tamanho do pool
 - ❑ Separar a tarefa a ser executada da mecânica de criação da tarefa permite diferentes estratégias para a execução da tarefa
 - ▶ ieTasks pode ser agendado para ser executado periodicamente
- ❑ A API do Windows oferece suporte a pools de threads:

```
DWORD WINAPI PoolFunction(AVOID Param) {  
    /*  
     * this function runs as a separate thread.  
     */  
}
```





OpenMP

- ❑ Conjunto de diretivas do compilador e uma API para C, C++, FORTRAN
- ❑ Fornece suporte para programação paralela em ambientes de memória compartilhada
- ❑ Identifica **regiões paralelas** – blocos de código que podem ser executados em paralelo

#pragma omp paralelo

Crie tantos threads quantos forem os núcleos

```
#pragma omp paralelo para
    for(i=0;i<N;i++) {
c[i] = a[i] + b[i];
}
```

Executar loop em paralelo

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }

    /* sequential code */

    return 0;
}
```





Despacho Grand Central

- ❑ Tecnologia Apple para sistemas operacionais Mac OS X e iOS
- ❑ Extensões para linguagens C, C++, API e biblioteca de tempo de execução
- ❑ Permite a identificação de seções paralelas
- ❑ Gerencia a maioria dos detalhes do encadeamento
- ❑ O bloco está em “`^{} - ^{ printf("Eu sou um bloco"); }`”
- ❑ Blocos colocados na fila de despacho
 - ❑ Atribuído ao thread disponível no pool de threads quando removido da fila





Despacho Grand Central

- Dois tipos de filas de despacho:
 - serial – blocos removidos na ordem FIFO, a fila é por processo, chamada **fila principal**
 - ▶ Os programadores podem criar filas seriais adicionais dentro do programa
 - simultâneo – removido na ordem FIFO, mas vários podem ser removidos ao mesmo tempo
 - ▶ Três filas em todo o sistema com prioridades baixa, padrão e alta

```
dispatch_queue_t queue = dispatch_get_global_queue  
    (DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);  
  
dispatch_async(queue, ^{ printf("I am a block."); });
```





Problemas de encadeamento

- ❑ Semântica das chamadas de sistema
- ❑ Manipulação de sinais
 - ❑ Síncrono e assíncrono
- ❑ Cancelamento de thread do thread alvo
 - ❑ Assíncrono ou diferido
- ❑ Armazenamento local de thread
- ❑ Ativações do Agendador





Semântica de `fork()` e `exec()`

- `fork()` duplica apenas o thread de chamada ou todos os threads?
 - Alguns UNIXes têm duas versões de `fork`
- `exec()` geralmente funciona normalmente – substitui o processo em execução, incluindo todos os threads

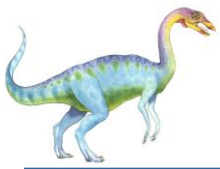




Manipulação de sinais

- n **Sinais** são usados em sistemas UNIX para notificar um processo de que um evento específico ocorreu.
- n Um **manipulador de sinais** é usado para processar sinais
 1. O sinal é gerado por um evento específico
 2. O sinal é entregue a um processo
 3. O sinal é manipulado por um dos dois manipuladores de sinal:
 1. padrão
 2. definido pelo usuário
- n Cada sinal tem **um manipulador padrão** que o kernel executa ao manipular o sinal
 - | **O manipulador de sinais definido pelo usuário** pode substituir o padrão
 - | Para thread único, sinal entregue ao processo





Manipulação de sinais (cont.)

- n Onde um sinal deve ser entregue para multithread?
 - | Entregar o sinal ao thread ao qual o sinal se aplica
 - | Entregar o sinal para cada thread no processo
 - | Entregar o sinal para determinados threads no processo
 - | Atribuir um thread específico para receber todos os sinais do processo





Cancelamento de Tópico

- ❑ Terminar um tópico antes que ele termine
- ❑ O tópico a ser cancelado é **o tópico alvo**
- ❑ Duas abordagens gerais:
 - ❑ **O cancelamento assíncrono** encerra o thread de destino imediatamente
 - ❑ **O cancelamento adiado** permite que o thread de destino verifique periodicamente se deve ser cancelado
- ❑ Código pthread para criar e cancelar um thread:

```
pthread_t tid;  
  
/* create the thread */  
pthread_create(&tid, 0, worker, NULL);  
  
. . .  
  
/* cancel the thread */  
pthread_cancel(tid);
```





Cancelamento de Tópico (Cont.)

- A invocação do cancelamento do thread solicita o cancelamento, mas o cancelamento real depende do estado do thread

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- Se o thread tiver o cancelamento desabilitado, o cancelamento permanecerá pendente até que o thread o habilite
- O tipo padrão é adiado
 - O cancelamento só ocorre quando o thread atinge o **ponto de cancelamento**
 - ▶ Ou seja, `pthread_testcancel ()`
 - ▶ Em seguida, o **manipulador de limpeza** é invocado
- Em sistemas Linux, o cancelamento de thread é tratado por meio de sinais





Armazenamento local de thread

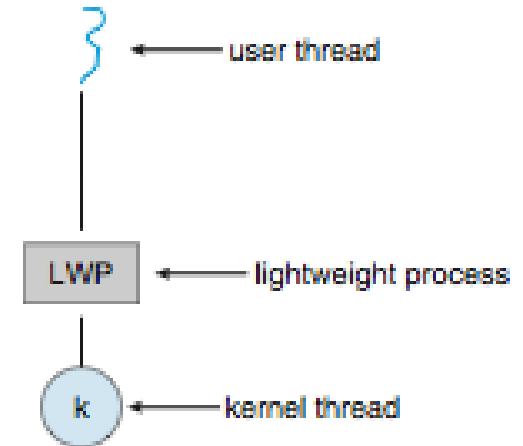
- ❑ O **armazenamento local de thread** (**TLS**) permite que cada thread tenha sua própria cópia de dados
- ❑ Útil quando você não tem controle sobre o processo de criação de threads (ou seja, ao usar um pool de threads)
- ❑ Diferente das variáveis locais
 - ❑ Variáveis locais visíveis somente durante a invocação de uma única função
 - ❑ TLS visível em invocações de função
- ❑ Semelhante a dados **estáticos**
 - ❑ TLS é exclusivo para cada thread

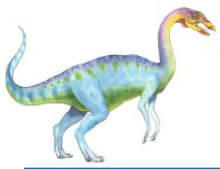




Ativações do Agendador

- ❑ Os modelos M:M e de dois níveis exigem comunicação para manter o número apropriado de threads de kernel alocados ao aplicativo
- ❑ Normalmente, use uma estrutura de dados intermediária entre os threads do usuário e do kernel – **processo leve (LWP)**
 - ❑ Parece ser um processador virtual no qual o processo pode agendar a execução do thread do usuário
 - ❑ Cada LWP anexado ao thread do kernel
 - ❑ Quantos LWPs criar?
- ❑ As ativações do agendador fornecem **upcalls** - um mecanismo de comunicação do kernel para o **manipulador de upcall** na biblioteca de threads
- ❑ Esta comunicação permite que um aplicativo mantenha o número correto de threads do kernel





Exemplos de sistemas operacionais

- Tópicos do Windows
- Tópicos Linux





Tópicos do Windows

- ❑ O Windows implementa a API do Windows – API primária para Win 98, Win NT, Win 2000, Win XP e Win 7
- ❑ Implementa o mapeamento um-para-um, nível de kernel
- ❑ Cada thread contém
 - ❑ Um ID de thread
 - ❑ Conjunto de registros representando o estado do processador
 - ❑ Pilhas de usuário e kernel separadas para quando o thread é executado no modo de usuário ou no modo kernel
 - ❑ Área de armazenamento de dados privada usada por bibliotecas de tempo de execução e bibliotecas de vínculo dinâmico (DLLs)
- ❑ O conjunto de registros, pilhas e área de armazenamento privada são conhecidos como **contexto** do fio





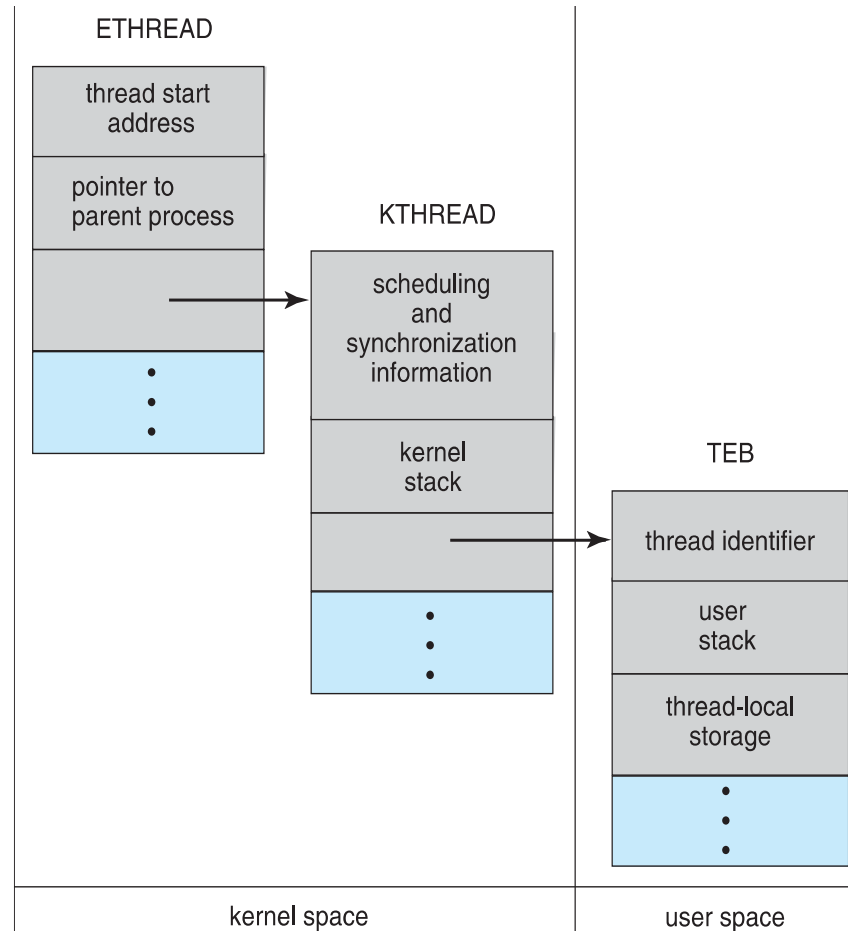
Tópicos do Windows (continuação)

- As principais estruturas de dados de um thread incluem:
 - ETHREAD (bloco de thread executivo) – inclui ponteiro para o processo ao qual a thread pertence e para KTHREAD, no espaço do kernel
 - KTHREAD (bloco de thread do kernel) – informações de agendamento e sincronização, pilha do modo kernel, ponteiro para TEB, no espaço do kernel
 - TEB (bloco de ambiente de thread) – ID de thread, pilha de modo de usuário, armazenamento local de thread, no espaço do usuário





Estruturas de dados de threads do Windows





Tópicos Linux

- ❑ O Linux se refere a eles como ***tarefas*** e não como ***threads***
- ❑ A criação do thread é feita através de `clone()` chamada de sistema
- ❑ `clone()` permite que uma tarefa filha compartilhe o espaço de endereço da tarefa pai (processo)
 - ❑ As bandeiras controlam o comportamento

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

- ❑ **estrutura estrutura_de_tarefa** pontos para processar estruturas de dados (compartilhadas ou exclusivas)



Fim do Capítulo 4

